WEEK 1 · LESSON 2 OF 24 · HANDOUT

Activation Functions and the Forward Pass

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=CqOfi41LfDw — open the lesson online to watch inside the course.

🎓 **Go deeper:** 3Blue1Brown: But what is a neural network? — https://www.youtube.com/watch?v=aircAruvnKk

🎓 **Go deeper:** MIT 6.S191: Introduction to Deep Learning (2024) — https://www.youtube.com/watch?v=ErnWZxJovaM

Your goal in this lesson: Compare the sigmoid, tanh, ReLU and softmax activation functions, explain why networks need them, and run a complete forward pass that turns inputs into class probabilities.

Why activations matter

Without an activation function, a layer is just a weighted sum — a straight-line function. Stacking ten straight-line layers still gives a straight line, so the network could never learn a curved pattern like the two moons. The activation adds a **bend**. With enough bends, a network can draw almost any shape.

```
import numpy as np

rng = np.random.default_rng(1)
W1, W2 = rng.normal(size=(2, 3)), rng.normal(size=(3, 1))
x = rng.normal(size=(4, 2))

two_linear_layers = (x @ W1) @ W2
one_linear_layer = x @ (W1 @ W2)
print(np.allclose(two_linear_layers, one_linear_layer))   # True: no extra power
```

Two linear layers are exactly the same as one. The activation is what makes depth useful.

The four you must know

```
import matplotlib.pyplot as plt

z = np.linspace(-5, 5, 200)
sigmoid = 1 / (1 + np.exp(-z))
tanh = np.tanh(z)
relu = np.maximum(0, z)

plt.plot(z, sigmoid, label="sigmoid")
plt.plot(z, tanh, label="tanh")
plt.plot(z, relu, label="ReLU")
plt.axhline(0, color="grey", lw=0.5)
plt.legend()
plt.title("Activation functions")
plt.show()
```

Activation	Output range	Where it is used
Sigmoid	0 to 1	Output layer for yes/no (a probability)
Tanh	−1 to 1	Older networks, inside LSTMs
ReLU = max(0, z)	0 to infinity	**Default for hidden layers** — simple and fast
Softmax	0 to 1, all outputs sum to 1	Output layer for several classes

Why is ReLU the default? It is very cheap to compute, and for positive inputs its slope is always 1, so learning signals pass through many layers without fading (Lesson 10 explains this "vanishing gradient" problem).

Softmax: scores into probabilities

For a problem with several classes (digits 0–9, or diseases A, B, C), the last layer gives one **score** (called a **logit**) per class. Softmax turns them into probabilities that add up to 1:

> $softmax(z_i) = e^{z_i} \div (e^{z_1} + e^{z_2} + \dots)$

```
def softmax(z):
    e = np.exp(z - z.max(axis=-1, keepdims=True))   # subtract max for safety
    return e / e.sum(axis=-1, keepdims=True)

logits = np.array([2.0, 1.0, 0.1])
p = softmax(logits)
print(p.round(3), "sum =", p.sum().round(3))
```

The scores 2.0, 1.0 and 0.1 become probabilities of about **0.66, 0.24 and 0.10**. Subtracting the maximum first does not change the answer but prevents overflow when scores are large.

A complete forward pass

The **forward pass** sends inputs through every layer to the output. Here is a network for 3 classes with 4 inputs, one hidden layer of 8 ReLU neurons, and a softmax output:

```
rng = np.random.default_rng(42)
X = rng.normal(size=(6, 4))                          # 6 examples, 4 features

W1, b1 = rng.normal(0, 0.5, (4, 8)), np.zeros(8)     # input -> hidden
W2, b2 = rng.normal(0, 0.5, (8, 3)), np.zeros(3)     # hidden -> output

hidden = np.maximum(0, X @ W1 + b1)                  # ReLU
logits = hidden @ W2 + b2
probs = softmax(logits)

print(probs.round(2))
print("predicted classes:", probs.argmax(axis=1))
print("every row sums to 1:", np.allclose(probs.sum(axis=1), 1))
```

`argmax` picks the class with the highest probability for each example.

Choosing the output layer

Task	Output layer	Example
Predict a number	1 neuron, **no** activation	Rent, temperature
Yes / no	1 neuron + **sigmoid**	Loan default, spam
One of several classes	1 neuron per class + **softmax**	Digit 0–9, crop disease

Key terms

- **Activation function** — adds a non-linear bend after each weighted sum.
- **ReLU** — max(0, z); the standard hidden-layer activation.
- **Logit** — a raw class score before softmax.
- **Softmax** — turns logits into probabilities that sum to 1.
- **Forward pass** — computing the output from the input, layer by layer.

Remember this: Activations give networks their power to bend; ReLU is used inside, and sigmoid or softmax at the output turn scores into probabilities.

Name: _____ Date: _____

Activity

Task 1

Apply softmax to [5, 5, 5]. What do you get, and why?

Task 2

Multiply the logits [2.0, 1.0, 0.1] by 10 and apply softmax again. What happens to the probabilities?

Task 3

Replace ReLU with tanh in the forward pass. Do the predicted classes change?

Task 4

Plot the slope (derivative) of the sigmoid: $s \times (1 - s)$. Where is it largest, and how small is it at $z = 5$?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why does a network need activation functions?

   A. To store data
   B. To remove the bias
   C. Without them, stacked layers are still one straight-line function
   D. To make it faster

2. What is ReLU(−3)?

   A. −3
   B. 3
   C. 0
   D. 1

3. Which activation suits the output of a 10-class digit classifier?

   A. Softmax
   B. ReLU
   C. Tanh
   D. No activation

4. What do softmax outputs always add up to?

   A. 1
   B. 10
   C. It depends
   D. 0

5. Which output layer suits predicting monthly rent?

   A. One neuron with no activation
   B. Softmax
   C. Sigmoid
   D. ReLU over 10 neurons

6. What is the forward pass?

   A. Plotting the loss
   B. Splitting the data
   C. Updating the weights
   D. Computing outputs from inputs layer by layer

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.